

00:00

As a data scientist, you would need to spend time finding your data and then cleaning it, preparing it, and making it ready for your analysis. So finding data is part of the process. You just spend time looking online or looking or maybe contacting different organizations to be able to get access to the data that you need for your analysis. And in this lecture, we'll talk about how you can find and import data of different types in Python. So these are the things that we're going to more or less cover, use the pandas read methods to import data into a data frame. I've seen some examples of that before. Download the file to disk before importing it into a data frame. Unzip a zip file to access the file that it contains. Use a SQL or SQL query to import database data into a data frame. Use the metadata of a stata file to analyze the data so we can see examples of reading data from the outputs of other tools like the tools that are using in statistical analysis they generate some output or they have certain data formats that we can actually import into our data frames and work with them. So we'll see examples as well. And also look into how to read JSON files. JSON is a very popular format for formatting textual files. It has a hierarchical format that allows embedding objects. So we can look into that and see how we can also read JSON files and more. So before we start, the question is what kind of data?

02:31

So our main focus would be on structured data. And by structured data, we mean data that comes in the format of a table, a set of rows and columns where each row specifies a data point and each column specifies the person's age, gender, salary, whatever the attributes of the person are, depending on what the data needs to capture. So we're looking at the structured data. That's our focus. That's our focus. That's our focus. Data can also come in the form of multi-dimensional arrays like matrices. That's also structural. That's also structured data. But you can, if you look at a two-dimensional array or a matrix, you can always convert that to tables. And you may also have multiple tables with contains information about, let's say, news tweets. So each row is a tweet that is, for example, published by a news publisher. And so each data point is a news tweet that has been posted on Twitter, for example, published by a news publisher. And so each data point is a news tweet that has been posted on Twitter, for

example, which also contains some geographical information, for example, the country of the country from which that, you know, that tweet has been posted. So what you can do is that you can create two different tables, one about tweets and one to capture the. Geographical information in this case, for example, the country, and then you can create a link between these two. So essentially you can have something like this. I have a table here where each row specifies a tweet. And. Each column specify one of the attributes of that tweet, for example, who has published this tweet or who has submitted this tweet on Twitter and so on and so forth. So one of the attributes could be the country, for example, which country is this coming from. So you might want to capture additional information about the country, but it will be redundant if you keep repeating them for every row. OK, so the way databases are designed is that you would actually create a separate table that captures information about countries. And then you can have different country codes, for example, U.S. Let's say the first one's country is U.S., the second one's U.K. So you can have, here you can have the country codes as the keys. So you can have U.S. Let's say this is just the title of the column. So it's country U.S. And you can have another row that is U.K. The corresponds to U.K. And then you can capture additional information about these countries. For example, you can maybe there is a display name, the format of the display, however you want to display the name of the country. You can specify it in this column and then maybe the population of the country, maybe the country's flag, any other information that you want to capture. You can embed that into this table. And then you can relate these two tables in such a way that the primary key in your country table, example, U.S., would be the foreign key, they call it foreign key in the other table. So basically, there is a reference from this table to this table that shows the relationship, shows the connection. So you can actually pick any of the country codes from, let's say, tweets table and find and then map it into the country's table using the foreign key, which is the country code to extract further information about the country. So there could be a relationship between the tables. This is what we mean by multiple tables. Sometimes you have more than one table. So as you can see, this is structured data. It says the rows and columns, a tabular format, and you have another table, and there is a connection, there's a relationship between them.

08:19

When we talk about structured data, we can also refer to time series, where they could capture evenly or unevenly events. So where there is a regular interval for the even

ts to happen that is evenly spaced. Then when the intervals are not regular, then that's on in that we refer to that as unevenly spaced. So this is just to give you an overview of what kind of data are considered structured, but we'll see examples of this. So don't worry if you get 100%. Continuing on a structured data, many databases can be transformed into a structured data that is more suitable for analysis. Structured data are preferred because they're easier to analysis. And also, it makes interoperability between different applications easier. Sometimes you have different software products that need to work together to achieve a certain objective. And having formats that can be easily converted to one another is preferred. Ideally you would want to have one standard format that you know works across all different platforms and software but that's not the case in real world for various reasons. Different applications take different inputs with different formats and converting different formats to one another is inevitable. So dealing with the structured data makes it easier to do that kind of things. So the thing is that we can sometimes, excuse me, transform data that is not structured into structured data. And sometimes it is not possible to extract, to convert the whole unstructured data to structured, but you can convert or you can transform part of it into structured, right? Or we can at least transform some metadata about the unstructured data into a structured format to at least know what kind of information are captured by, you know, that, for example, a file that contains unstructured data. for example, a file that contains unstructured data. And by unstructured, we mean a structure that is not known. Sometimes, of course, every file has some structure, but if the structure is not popular, is not known, it cannot be easily formatted under one of the known types of data, one of the known formats. We refer to it as not structured, structured, as opposed to structured. So, excuse me. So if it is not possible to transform structured data, sorry, transform data that are not structured into a structure, it may be possible, as I mentioned, to extract some of the features of them and make the transformation. For instance, a collection of news articles can be processed into a word frequency table, which can then be used for NLP or natural language processing. That includes, for example, sentiment analysis. So you can convert or transform a set of news articles in a table that contains the words and their frequencies. And then you can you can use that for further language processing or natural language processing. further language processing or natural language processing.

12:52

All right, so now let's look at some of the, some different types of data or data types.

Numerical data. So numerical data is a numbers, consists of numbers such as height, weight or age, and they can be continuous or discrete. Continuous numerical data can take any value within a certain range. And that is, for example, an example of that is temperature. So temperature is continuous. And discrete numerical data can only take certain values. So they cannot take any values within the range. They can take only certain values within the range. And that is an example of that is, for example, number of pets. So you cannot have 1.5 pets in real world. You can only have either one or 2 or 3 or 4 you cannot have 3.2 you cannot have 6.8 so these are discrete numbers so continuous numerical data as opposed to discrete numerical data and numerical data can be analyzed using regression analysis, correlation analysis, and hypothesis testing. So you can apply these techniques for analyzing numerical data. Okay, so the next group is, or the next type of data is categorical data. Categorical data consists of categories or groups such as gender, race, or occupation. So these are not numbers, these are categories. So somebody's gender is not a number unless you want to represent them as numbers, which is possible, but you're not required to. So you can specify people's gender, race, occupation, different categorical data in terms of the category they belong to. This data is often they belong to. This data is often represented using labels or strings and then include categories or labels that describe certain characteristics or attributes of an object or event. We have two types of categorical data, nominal and ordinal data. So nominal data are the ones that for them order doesn't matter. So there's no order or particular rank among different values for the data. An example of that is colors, for example. Colors do not have certain priorities, a certain order or rank among them. Another example, an example for ordinal data, however, is, for example, education level. So education levels come in order. There is a sequence, there is an order that you can specify. So they're both different examples of categorical data. Although they are not numerical, but if they are ordinal, there is still some order among them. So if there are numbers, of course there's order. You can always say five is greater than four. One is less than 10. There's order among numbers, so you can compare them based on which one's smaller, which one's larger. For categorical data, if they are of type nominal, you can't do that. Example of that is colors. You cannot say one color is greater than the other, unless you have a definition for that. That's a different story. And for ordinal data, you can compare them based on the rank order. Categorical data can be analyzed using contingency table analysis, logistic regression, and chi-square test. So these are the three methods or techniques that can be used to analyze categorical data.

17:42

Contingency table is a table that basically allows you to find information or capture information about the frequencies or the occurrences of or the relationships between two variables. So you can, for example, have, let's say you have, what do you call it, two people who are running for presidency. So presidency candidates, candidate one, candidate two. And then you can have, for example, male and female, and you can capture how many people, for example, have voted, how many male have voted for C1, how many female have voted for C1, how many male have voted for C2, how many female have voted for C2, and then you can sum them up. So you can have how many, here you can have sum of these two values. So how many male have voted for C1 and C2. Or in general, you can say how many male have voted because this contains both people who have voted for C1 and the ones who have voted for C2. And same, you can do the computation for female ones. So you can sum these two values up, should give you a new value that shows how many female have voted in the election. And then you can have another column here that shows how many people, that means male and female in general. In this case, we are two categories for gender. how many people that is male and female in general. In this case, we are two categories for gender. So male and female, how many have voted for C1? So you can capture it here. That will be the value that goes there. And then you can have how many male and female together have voted for C2 and get the value here. And yeah, so you can capture, and then you can have total number of the people here. This would be your, what do you call it? Contingency table. And this would be your labels. You can have C1, C2 here. And you can just write the, put the labels that shows what was the description of each column and the sum of blah, blah. So M and F goes here. And then you can of course compute the marginal distributions based on that contingency table. We've included a video that explains this. You can watch that for further details. You wanna see some examples. This was just to give you an overview of what we mean by now. So as you can see, it shows the relationship between two categorical variables. So in this case, you know, the candidate for presidency would be C1 or C2, the two different people. It's categorical. And male and female are also categorical. And they're both of nominal type because there's no order between them. You cannot compare them in order, not compare male and female in order or two different candidates in order. So yeah, so this is an example of contingency table. And then of course you can also use logistic regression. So logistic regression is a form of classification essen

tially, which allows you to decide about the value of a variable that is either true or false, binary, but that will be specified based on the values of a set of other variables. For example, you can say if someone's gender is male and, you know, they have had this amount of income and, you know, this amount of this level of education. So you can have a combination of numerical and categorical data, data types or variables. And then you can say, for example, if someone has these characteristics, they are likely to, for example, succeed in something. I can't think of something right now. So you can just decide about the true or false, decide about the value of a variable that is either true or false. So essentially logistic regression is used for machine learning purposes for classification. I've also included the video that further explains that. And of course, class score test is used for hypothesis testing. Once you look at your data, sometimes you find certain patterns and that pattern is only a hint. It only suggests that what you're seeing could be the case, but you don't know. So to kind of test that or examine that, you use hypothesis testing. You have a hypothesis and you want to test it, you do hypothesis testing. And then to test your hypothesis, you use different statistical tests, different levels of confidence, depending on how confident, tests, different levels of confidence, depending on how confident, how confidently you want to report your findings. And then you can either accept or reject the hypothesis at that particular confidence level. So that's the purpose of hypothesis testing, which is sort of in differential statistics.

24:39

So another data type that we might deal with is time series data. So that is data that is collected over time, such as stock prices or weather measurements. This data can be used to analyze trends, patterns or seasonality of the data. And this type of data is called is collected over time at regular or irregular intervals. So we talked about this before, previous slides. And time series data is often used to analyze trends, patterns, and seasonality, as we discussed. Time series data can be analyzed using moving average trend analysis and seasonal decomposition. These are the three different techniques that can be used for analyzing time series. We'll talk about time series later on when we talk about more advanced topic on analyzing data. So talk about systemology. Systemology refers to the pattern of regular and predictable fluctuations in a time series data that occur at regular intervals or over a specific period of time usually caused by a combination of factors, such as weather pattern, holidays, cultural events, and business cycles. For example, you can look at the traffic, if you can formulate traf

fic based on some data over time, you can see that, for example, the congestion in certain areas, for example, around the stadium increases whenever there is a,, increase s whenever there is a match going on, something like that, whenever there's a game. So usually you can associate these spikes with a particular event. So seasonality is commonly observed in various types of data, including sales figures, customer traffic. Customer traffic means the traffic on a website, for example, or the traffic in the actual shop, in a physical shop. So either way can monitor the traffic of your customers. So website traffic, as I mentioned, and stock price by identifying and understanding the patterns of seasonality in the data. Businesses and analysts can better predict future trends. So that's what we do when we do time series analysis to be able to predict the future outcomes. For the very variables of interest. Another type of data that we deal with is text data. So text data includes unstructured text, such as, for example, customer reviews or social media posts. I gave you an example earlier about the tweets. So the tweet itself could be structured, but the tweet text, that means the text that the person has actually tweeted, is not structured. It's just text. But if you, what do you call it, capture the tweet information, for example, say every tweet has a published date. It has an honor, for example. It has the country of origin. It has also it has a text. What is the text of this tweet? So the text itself is not structured. is not structured. So it can be analyzed using, that means text data can be analyzed using natural language processing or NLP. And so whenever you see NLP, it often refers to natural language processing. And examples of that is topic modeling, sentiment analysis are used for topic modeling, for example, is used to identify different topics in the text. For example, you might you might collect a set of Twitter tweets or Twitter messages and then try to find what topics these tweets are talking about. And you want to identify different topics that these tweet messages relate to. So for that, we carry out topic modeling and we use topic modeling techniques such as, for example, the LDA topic modeling approach. It's the classic one that works pretty well for many, many real world examples. Of course, there are more sophisticated models that use deep learning and so on and so forth. So topic modeling is an example of carrying out an NLP operation or task on unstructured data. And throughout the process of course, we transform the unstructured data to structured to be able to work with that. So yeah, another interesting example is the usage of, Yeah, another interesting example is the usage of, again, the concept of NLP, but for medical NLP. unstructured data, in this case text, is used in the medical context. So watch that video when you have time and you see some interesting examples there. So again, there are unstructured data and one exa

example of unstructured data is text. And text can be transferred to structured data. For example, when you have the frequency of the words listed and you work with that for sentiment analysis, give you the example of it earlier. And as I mentioned, we use NLP techniques, so natural language processing techniques to analyze text. All right. So that's a whole area of research. I just briefly mentioned NLP. But, you know, a lot of people who actually do research on that, they work on improving the techniques for sentiment analysis, the techniques for sentiment analysis, emotion analysis, identifying the tone in the messages, identifying the topics, topic modeling. So each of these areas are big areas of research and people are working on them.

32:00

Okay. Another type of data is spatial data that correspond to the data that are collected from devices such as GPS or satellites. So that includes geographic or location-based data such as GPS coordinates or satellite imagery. or satellite imagery. And this information can also be analyzed using, they can be analyzed using statistical methods. Some of them we already discussed, but sometimes you need to analyze them in combination with other data that relates to this data. So you can actually make more sense of data. For example, you need GIS to be able to make sense of the coordinates that you're receiving from your GPS, for instance. So you might have, for example, a company that, you know, works on fleet management, right? And what it does is that it collects information about their different vehicles and then analyzes that data to provide advice or make recommendations for, for example, better fleet management. Right. So that's such a software can use data analysis as one of its components to provide recommendations. And the type of data that one of the types of data that it deals with is a spatial data that corresponds to the geographical and the geographical location based data that is, for example, the location of the cars. And of course, you would need a GIS to understand where actually they are on the map and be able to, for example, make recommendations for improving your vehicle management system, for example, something like that. So spatial data can be analyzed using techniques such as geographic information systems and spatial statistics. So spatial statistics is built on top of the standard methods in statistics. They utilize those methods for analyzing geographical patient-based data. If you want to know more about spatial statistics, you can watch this very interesting video that I've shared here from one of my former colleagues from the University of Wollong. Interesting too. former colleagues from the University of Wollong. And it's interesting to learn about this. Okay. Image

data. So another type of data that have done in my deal is image data. Obviously, there are lots of applications for that, some of them with privacy concerns. So essentially, you can use computer vision to utilize image recognition for detecting people's face, perhaps detect a suspect, or perhaps detect someone, maybe if you think more positively, you can use this, for example, to detect someone who might be in danger, a person who might offend while they're walking on the street because they might be suffering from some medical conditions or they may just, you know, fall down or something happens to them. So if you see someone in a if you can use computer vision to detect the person who is in danger, they're trying to cross the street and, you know, they may not be they may not look all right. Then maybe someone can go and help them or, you know, you can think about different applications. And as I mentioned, there's also always discussion about the privacy concerns for using such technology that keeps monitoring people in public. But, yeah, they're widely used and there are so many applications. And as I said, computer vision is one application for image processing. So image data includes images or videos. So we're not only referring to pictures. Of course, when we talk about videos, we are talking about a stream of images, right? So video is a stream of images that are put together and so on. You might or might not have audio with it as well, but yeah, so video is classified under image data here for this reason. Deep learning has proved efficient actually in image processing and computer vision and there are many interesting applications of deep learning in this area so you can have a look there are many publications and real world applications that you can use and use classifiers for detecting images. So this classifier is a machine learning models that are used so that they can help you, for example, identify different animals. For example, you might be given a set of different animals, including dogs, cats, horses, different animals. And then the classifier can tell which one's dog and which one's not the dog. So this or things like that, we can tell you which it can label each animal based on their category or their breed.

38:11

Audio data. So talking about image data, you can also process audio data, audio data. We have seen many examples of that. many examples of that where we may have used some of these virtual assistants like Siri or Cortana or others that are available. So these are these applications use audio data, they process audio data or they process voice and audio data is not necessarily voice it could be other things could be for example you might have used one of those applications that measures the noise level

el where you are so you can use that for example measure you know how noisy an area is but it's normal or not so you can use it for different applications, different purposes, whatever the type of the audio is, you can use them for doing some interesting things on music, but you can also use audio data or processing audio data for speech recognition or voice recognition. And examples of that, of course, as I mentioned, virtual assistants that I can use and so on and so forth. So all your data can be analyzed using techniques such as signal processing and Fourier analysis, mathematical methods that I used to model voice based on the frequency domain, time domain, and perhaps convert them to one another depending on the application. So Fourier analysis and signal processing are commonly used for analyzing voice or audio. And what else? Okay. One more time. Logistic regression and we have another video that talks about yeah, contingency table as I discussed. So I'm going to... Let's watch this video together. Excuse me. I'm going to take a break, have some water while you're watching this, and then we'll continue. So I'm just going to pause at my side, and I give you some time to watch it at your side and then we'll continue. Yeah, so you can watch this other video as I mentioned on contingency tables. It gives you a simple example. But the next thing we want to cover is how to find the data. So we talked about different types of data. Now we want to focus on how to find data. You can find data from various sources. That could be either internal, or that is through internal data sets. Let's say you're working in an organization and there are some data sets in your organization, you can use them. Or they're from third parties or third party websites. You can go to websites such as kegel.com or datasetsearch.research.google.com or registry.opendata.aws. dataset, dataset search, research.google.com, or registry.opendata.aws. So these are the three sources that you can find, lots of openly accessible datasets, and you can start working with them. So your Avocado dataset that you're using for your tasks is coming from Kaggle.com. And if you go there, you can see several examples of people using them and some discussion about the data. So if you're interested, you can go there and follow that. And of course, you can also see examples of the things that people are doing to analyze these datasets. And, of course, you can also see examples of the things that people are doing to analyze these data sets. Right, so our data can come from a bidding organization or from third parties. Third parties could be websites, or sometimes you need to purchase the data. So that could be another company that owns the data, and you might need to purchase a license to be able to use it. But as I mentioned, there are lots of free data sets that you can access from the links provided that you can download and work with.

04:18

Sometimes you need to use an API to get access to the data. API stands for application programming interface. Uh, you can think about it as a kind of a data block you connect your application that means the software that you are writing your code your Python code into another software that has been written by the data provider. Okay, so you're dealing with two software components here. One is the provider and one is the user, which is you. So the data analyst is the user, which is you. So the data analyst or the user, we use the provider's API. So the API is like a gate here. Okay, so the user uses the API, this is your API here. Uses the gate to get access to the data of data sets that are being hosted by the provider. OK. So why is this important? Because a lot of the data that you can find online come from APIs. For example, if you want to collect Twitter data, if Elon Musk loves you, then you can use a Twitter API to get access to Twitter data. And that is not as simple as I said. That requires you to understand how the API works, to understand the protocol, in other words, to understand how the data is being provided, what is the communication protocol with the API. Okay, so you need to fully understand or to understand it to extent that you need it. So you need to understand the API to any extent that you need to know to be able to talk to the provider, to the software that has the data. So your software needs to talk to the API to get the data from the provider. And the provider itself could be another software, right? And as I said, you can use Twitter API or YouTube API or other social platforms APIs. And sometimes the organizations create APIs to allow other applications to interact with their data sets, databases or with their software. Sometimes these APIs are restricted to certain organizations. For example, let's say maybe a government organization creates an API on some geographical information about different states, and they might want to share it with some other organizations to do certain research, for example, on different types of, for example, farming products, right? So one organization can allow another one to use their data. And sometimes they specify, you know, some protocol and, you know, some terms and conditions that, you know, you are allowed to do these kind of things with the data. You're not allowed to do, you know, other terms and conditions that, you know, you are allowed to do these kind of things with the data. You're not allowed to do other things with the data. So there are also ethical aspects involved. There are also privacy concerns involved, security concerns and all that. So getting access to data sometimes would be very complicated. And getting access to data sometimes would be very complicated. And it's not as easy

as, you know, just plugging to the API and get the data nice and ready and work with it. It could be a sophisticated process and it could be time consuming. But sometimes it's also easy to use the APIs. For example, if you want to work with YouTube, you can easily create an API key and start grabbing data from YouTube. Of course, there's a limitation, so you have a limited quota to grab data from YouTube, but you can do it otherwise. you can do it otherwise. You can extract data without having to pay for anything.

04:49

Right. So here's the thing. To work with the API, just to make it more complicated, you need to know programming. So that makes it difficult for data analysts who don't have strong programming skills. So they might need to get some help from someone who knows programming or have stronger programming skills to help them connect to the API and grab the data. Sometimes you just connect to the API and grab the data and that's a once done activity. You just do it and after that you're good to work with the data. But sometimes you need to keep reading the data from the API. For example, if you're analyzing the trends in the stock market or something like that, you need continuous data to be provided. You need data to be continuously provided to your application to be able to provide analysis that you are aiming for or you're required to do. and for all you're required to do. So in such cases, this becomes even more important to be able to actually build applications that can read data from different data sources to different APIs and prepare them and convert them or transform them into formats that can be used in your application. And that obviously is a lot of programming activities. Depending on the API, you may need to provide an API key or authenticate yourself. Well, API key is also a way of authentication, but distinguishing between API key and more a way of authentication, but we're distinguishing between API key and more sophisticated authentication methods. API key is as simple as, for example, you can go to your Google accounts and request access to different APIs that are available there, for example, YouTube API, and you can create credentials for yourself. And the type of credentials that you available there, for example, YouTube API, and you can create credentials for yourself. And the type of credentials that you can choose, you can use authentication or you can use API key. Then create an API key for yourself and then use the API key to connect, for example, to the API. So why is this important? So this is a security concern. This is for the data provider to make sure that they have some kind of control on who is reading their data, how they're

accessing their data, perhaps what they're going to do with their data, and also to prevent from their system being attacked. Because if they don't have any control, then people can unlimitedly send requests to their servers and, you know, their servers, it can keep their servers busy. So they might be targeted by, you know, denial of service attacks, different kind of security risks that can affect them. So this is for the data providers actually to protect the service and protect their data. So what you need to do is that you need to kind of say hi to them and say, you know, I'm going to use your service. And there's a process always for that. So you can just follow steps for every API and they have instructions to tell you, for example, to create an account and send a request. Sometimes if you're requesting, you know, some specific service, let's say you want to do research on Twitter data, you might have to wait for, you know, some time for them to actually read your case and to confirm that you're allowed to use the data and then grant you, you know, your authentication details, the key and, you know, whatever you need to authenticate yourself. And these keys are provided as the input of the API. So you would be sort of handshaking with the API by sending these credentials so they can recognize you and know who you are. You've already been granted access. You are a valid user. So from now you can start using the data. So that's the purpose of authentication. And again, this also requires some programming knowledge and you need to actually understand the protocols that are used by different API's and sometimes the API's are sometimes there are different versions of the API's and in each version the protocol changes so that can create some trouble because you need to learn the new protocol and see how the new protocol works. Some of the data that you could retrieve from the previous protocol or previous versions of the API, you can no longer access them or you have to access them in a different way because they have changed the structure of the data. So this is not ideal, but it happens all the time. So this is another challenge that you face. You might spend hours learning and getting used to how to work with an API, and then all of a sudden you have to do something new. Most of the times the changes are not radical, but still they might affect many parts of your analysis or the data that you have been using in your analysis and is no longer available, you have to change that. So that's inconvenient. Getting the data via APIs.

54:21

Okay, so here's an example of an API, Twitter API, which we refer to it. We use Twitter API version 2. So if you search for Twitter API version 2 on Google, you can find this

page. And from there, you can look at the documentation, look at how to get started, how to get access to the data. You can see examples of how queries can be sent to the API and examples of sample data that can be retrieved from Twitter. So you can see many things there if you're interested. Another API is, open the accessible API is YouTube API. Again, you can search for YouTube API and you can see the examples. Here's an example of sending a request to YouTube API and the response that has been received. You can see that there's a URL here that is sent to YouTube API and this is a description of what this does. So this example retrieves a video resource and identifies several resource parts that should be included in the API response. So essentially, as you can see here, this process, I want the snippet. I want the content details. I want the statistics. I want the status. So I want this information about the video. So here's an example of a request being sent to YouTube API, YouTube API version three, I believe. And this request, this kind of information about the video and this is the video ID. So we are asking for snippet, content details, statistics, status of this particular video. And, of course, we also need to include our key here and see that this part key equals your API key. Just cut. So basically, you need to also include your API key there, which could be different for every user. need to also include your API key there, which could be different for every user. So you need to insert your API key there and then send the send a query and you'll get what's here. And the API response is this. So the format that you can see here is JSON. We'll talk about JSON a bit later, but this is one of the formats that you can convert or read into a data frame. If your JSON file is one level, then you can easily read it into a Pandas data frame without doing much. But if it has multiple levels, that means if it has embedded objects, so it has, which is the case in the case of this JSON file, you can see that multiple levels. This is your root level. And then there's one level down. And then inside this, you have other levels. For example, you have snippet here. So you have multiple levels. So in that case, you need to do some more sophisticated analysis. You need to actually read this, iterate through these levels and extract the data that you need and then transform that data or those records of data or those data points into a data frame and use them. So it becomes a little bit more sophisticated but there are libraries to do that so difficult. It just requires some further analysis. So you need to decide which part of data you need and what you exactly want to capture and then do that. We'll give you more examples of these later. All right, so importing the data frame into a, sorry, importing the data into a data frame. So we talked about different types of data. We also talked about how to find data through internal sources, external sources, using APIs. Next, you want

t to import your data into a data frame. So you can directly import your data, or you can download the data, Going to a data frame. So. You can directly report your data or you can download the data and then important, depending on the structure of the data. And sometimes your data contains several files that are zipped together and you might want to first download them and then extract the zip file and then get access to all the files that are in the, or some of the files that are in the extracted folder. You could do that manually. That means you can go and just download the file and extract it on your drive and pick the folder that you need. Or you can do it through the software. and extract on your drive and take the folder that you need, or you can do it through the software. The number of the files increases when it becomes more sophisticated, it's easier. It would be more, it would make more sense to do it automatically through your software than it is through your Python code. But of course, if you just want to download one zipped data file, you can just simply download it, extract it, and then read it as a example CSV or whatever the format is. So you can directly import or download and import.

01:00:16

Common file formats for data, you have different types of data here, CSV, which is the comma separated values, stands for comma separated values. It's a text file where the values are separated by comma. And of course, there's a header row that specifies the titles or labels of the columns. And those are comma separated as well. You can also have TSE file, tab separated values. You can have Excel file, a starter file, a starter statistical package, JSON file, which I just showed you an example of a JSON file, the YouTube data. which I just showed you an example of a JSON file, the YouTube data. This is a JSON file. And this is JSON format can be saved as a JSON file. And that stands for JavaScript object notation, which has been primarily used in JavaScript. Of course, this is just a data format, which can be used across different programming languages including Python and it's widely used. There's also XML, stands for extensible markup language, and it has its own pros and cons, but that's another way to capture data. SAS, SPSS, there are statistical packages and HDF5, that is for large data, it's particularly useful for managing large data and capturing metadata about large data that is also used in Python. And the type of data also, the type of data file also, the type of data file could also be a zip file that may contain, you know, any other data types. You might have a zip file that includes several CSVs, for example, right? So, there's different formats of files. You can use Panda's methods for importing data into data f

rame. You can use read CSV, read Excel, SQL, JSON, HTML, clipboard to read data from the clipboard into a data frame, SADA, SAS, PICO, we talked about PICO before, Feather, and all that. Different types of data can be read using Pandas methods for importing data. You can see an example of, or more details here about reading a CSV file where you can see the parameters as well. In the previous example, you couldn't see the parameters. This is just a list of the functions. Of course, if you want to know more about them, you need to know their parameters and how you can use them, what kind of values you can provide to them. So here you can see that a read CSV file can get a file path or buffer as its input. That's file path or buffer, which is the file path or the URL of the CSV file to be read. In other words, it's the local path or the global path on the Internet. You can use either. Then there's a separator that tells you, you're telling the read CSV file how your data is separated, whether it's separated by comma, or if there's any other convention, you can customize that here. So read CSV file, so that read CSV function or method knows the convention or the protocol that has been used in the CSV file, whether it's comma separated or tab separated, for instance. But it is comma separated or tab separated, for instance. And then you have your next parameter, which is header. Then you set header to infer. It's going to pick the header by default. It's going to infer it. But if you want to set a specific row to be the header, you can also do that. You can just specify the row that you want to be used as the header column, or as the column header, sorry. What else? Names. So names is a list of column names. If you don't set it, it's just going to pick the ones that are coming from the header. Otherwise you can sort of rename them right away to the names that you provide here as part of names. You can also specify the, also specify the specify that which column which column to use as the index of the data frame and if you don't do that of course the default indexing is used otherwise you can say for example I want the student numbers for example, to be used as index. So these are the things that you can control when you read the data from a CSV file into a data frame. You can right away set these parameters or you can change them later. So, of course, you need to provide the file name, file name so you can read the data. But some of these and of course, the separate separator, but some of this can be changed later. So you can rename the rename the columns later in the data frame and change the indices later. So we can even postpone some of these changes. You don't have to do them right away, but this gives you the option. You can postpone some of these changes. You don't have to do them right away, but this gives you the option.

06:37

Another example here is `read SQL` or `read SQL`, which reads a SQL file or SQL database or a table from a database. So what it does is that you have a SQL parameter that is the query that you're going to execute on the database or on the tables of the database. A database is composed of different tables. A database, for example, can have a database of employees and a database of different tables. One of the tables could be employees. Another table could be salary details. Another table could be, you know, expenses. But an organization can have several tables that could be linked to each other based on different keys. And you might want to execute queries on these tables that are interrelated. And we call these type of databases relational databases. And we call these type of databases relational databases. So you have several tables that are interconnected, not necessarily all of them, but there could be a relationship between them. And different types of relationship, one to many, many to many, one to one. So there are different types that you can look if you're interested in this topic. But at this stage, I just want you to know that you can execute a query on a database and get some information, some data from the database. And for that, you need a connection object. So `CON` is the database connection object, such as for example, `SQL Ogn` or `Ngin`, or if you're using, for example, `SQLite`, or if you're using, there are different databases, of course, out there, `SQLite`, `MySQL`, `Oracle`, there are different ones, and you might need to create connection objects that allow you to connect to them. And then you use the connection object to do that. And then you can specify the index column that specifies which column to use as the index of the data frame again. So this is for the data frame that will be created based on the result that is being returned by the query. that is being returned by the query. So when you execute `read SQL`, it's going to read the, what do you call it? Execute the query and return a data frame. And you can then change the index. And this is of the data frame or set other parameters. For example, you can convert numeric columns to for example, floating data types, to floating data types if this parameter is set to true. And what else? Parse date, for example you can right away as soon as you read the data from the database if there is there is some column that contains some date information some dates as objects or strings you can actually parse them into date objects or datetime objects and process them into data objects, so datetime objects, and process them as data objects. As I said, some of these you don't need to do initially. You can always convert the columns to an appropriate form based on the application later on when you prepare your data. But it might be easier to do it right away if you already know what

t you're going to do and what format you exactly need. You can do it when you read the data from the, in this case, database, SQL database. Okay. So Pandas methods only work when data is in a tabular form. If the data is not in tabular format, then you might get error when you use the read functions or read methods. So the read JSON method may return error if your JSON file has more than one level, for instance. I mentioned earlier that if it's one level, then you can easily use Pandas read method. Otherwise, further analysis is needed, more sophisticated analysis is needed to choose what you want to extract from JSON file. As a result, PineOS method works best on tabular data or tabular files like CSV or Excel files, but they will fail on files with complex nested data. fail on files with complex nested data.

01:11:10

Importing a CSV file from a website. So we have seen examples of this. I'm just going to skip it. It's very straightforward. Just provide the URL and read the data from the online source, load it into CSV. And you can also read a specific sheet from an Excel file. In this case, our Excel file, we are using index zero here. Sheet name equals zero, which means the read Excel function, read Excel method is going to read this Excel file, but it's going to read only its first sheet, okay? In other words, this line will lead to reading the first sheet of this Excel file, which is stored locally, and it leads to reading that and loading it into a data frame. And here you can display the contents of the data frame or the first five rows of the data frame using the head function. And this is what you get when you execute this. Right? So this is not the list of whole columns. Three dots here. So the list continues. There are 30 columns actually in this dataset. So this data file, this xlsx file, this is an Excel sheet, Excel file. This is an Excel sheet, Excel file. Its first sheet contains 30 columns, and you don't know how many rows here because we just printed the first five. But you know how to get the number of rows. If you don't, please go back to the previous lectures on essentials for data analysis. And we have shown you how we can get the information about the data frame. Okay, downloading a file using the URL retrieve method. So URL retrieve comes from URL lea veleap.request module. So you need to import this. So from URL.leap, import request. This imports the request object. And then you can use request object to call the URL retrieve. And this is what we did here, what we have done here. object to call the URL retrieve. And this is what we did here, what we have done here. So request that URL retrieve, that takes a URL as its input and extract the data from that URL and stores it into a file that's been provided here in this case, PresidentPost2016.csp. All right.

So what it does, it reads the data from the URL provided here, and this URL specifies a CSV file. It's a full pass to the CSV file online. It reads the data. It grabs the data, and then it stores it into the file that's being provided here. So this is the name of the destination file, President Poles 2016.csv. So this would be on your local drive. Okay, so in other words, what these two lines do together, they download the contents of the CSV file for you. Okay. So you download the data, and the next step, obviously, is to read the data into a data frame, right? So `pd.readcsv` is going to read the data into the data frame, and you use this handle `Pulse` in order to get access to the data frame. In other words, `Pulse` is your data frame, simply said. So you use `polls` to print out or display the first two rows of the data. And again, we have 27 columns. Obviously not all of them are displayed, only some, just to give you an overview of data. But there are 27 columns and two rows because we have selected two rows to be displayed. So you can check and see how many rows are actually in the data, but we have 27 columns.

01:15:41

Download and extract a zip file. So here's an example of downloading from some online link. So here's the URL. And what we're doing is that we're going to download this, download this zip file from the URL provided here, and then extract it on our local drive. So let's see how we do that. So, again, we need to use `request` from `URLib`. So from `URLib` import `request`, `request` that URL retrieve, zip URL, file name equals file name. So what this does is that it grabs the zip file from the URL provided here, from zip URL, and then it stores it on your local drive. So, so far, we have not extracted anything. Okay. So far, we have just grabbed the zip file from this online resource and saved it just the same way we did in the previous example for the CSV file or the file for the spreadsheet. So we have so far saved it on local drive. The next step is, so we need this library. We need zip file object from zip file. So from zip file, import zip file. And then we're going to use this zip file to extract the file. So first we need to create an empty list to store the names of the extracted files. So `file_names = []`. Is this familiar to you? This is to call the constructor of lists. So we are creating, this will create a list for us and gives us a handle to that list and that handle is both file names. In other words, just to summarize that, this line creates a list and names it `file_names`. So we can use `file_names` to get access to that list. So, so far the list is empty. It's just a placeholder, but from now on, we're going to use it. Let's see how. So with zip file, so we're passing this path, which is the path to the zip file that's been stored on t

he local drive. And the mode of accessing the file is read. Okay, so these two things say, firstly, this is the file, this is the zip file, this is the path to the zip file, and I want to open the zip file in the read only mode. So I'm going to only read it. Okay, so with zip file, blah, blah, this is what it does. It just says, I want to open this file, this zip file, and in read mode, and that is as zip. So we refer to the file as zip. And then we call, just gonna remove this so you can see what's happening. And then the next step is that we use zip, which is a handle that refers to the file to extract the contents of the file. So zip.extractAll is going to extract the zip file there on the local drive, wherever the zip file was. Okay, so this path under the data folder. It's going to extract the contents, they will create, it would extract the zip file, and then the next step, iterate over each file in the zip file. So for file in zip.info list, so zip.info list, it will give you a list of the files. So zip.info list, it will give you a list of the files. A list of the files in the zip file. So you know how many files are there. So for every file in the zip file, what you're doing here, you are adding the file's name to the list of file names. So this is the empty list that you created earlier, file names. You're appending the name of the file to this list, the file names. So what file names is going to contain at the end of this for loop is the list of all files. Every time, for every file in the zip.info list that means for all the files we are adding their names at this line we're adding their names or appending their names to the list of the file names. Okay and then we are printing the file names, sorry, we are printing the file's name, compressed size, and the actual size. So the information about the file. So we grab the file name, we added it to the file names, and we are printing file.filename and file.compresssize and file.filesize. So we know what is the actual size of the file, what is the compressed size of the file, and what is file name. We print that for every file in the zip file. Okay. So use standards to read the Excel file at the specified path and create a data frame. So after we extract the contents of the file, so at this stage we have already extracted the contents, then we can go to that folder, for example, to this folder that contains the contents of the unzipped file, right? And one of the contents there is all data in 2018.xlsx, which is a spreadsheet. So basically, it just goes to the folder that we just extracted or the folder that is the result of the extraction of the zip file and reads the file that is in that folder and that file's name is provided here. Again, it sets the sheet name to zero, which means read the first sheet of this file and create, so this is a read Excel file, which is going to create for us a data frame from the first sheet of this Excel file, which is achieved through unzipping the zip file. And that data frame from now on can be used to, for example, display the head or do some whatever analysis we want to carry out. Download and extra

ct the zip file. So, all right, so continue on that. This is the output you get. So in this case, you only have this one file there, which is displayed here. So you have the file name, and then you have the compressed size, and then you have the original size. So you can see the compressed size is smaller than the original size. And you can see how much space you've saved by compressing the file. Or of course you haven't compressed it. I'm just saying that you can see the difference between the compressed size and the extracted size. And of course, the rest of this is the result of executing the head function that is going to print out the first five rows of your data frame. Okay, so this was an example of reading a zip file from an online source, extracting it on your local drive, and then iterating over the files that are in the zip file and extracting their metadata. That means their name, their size, their compressed size, their original size.

08:20

Okay. Getting database data into a data frame. So now let's talk about databases. Here we have an example. We are using SQLite 3. So this is the database type we are going to use, SQLite. As I said, there are many different types of databases out there. There's SQLite. There is Microsoft SQL Server, there's Oracle, there's MySQL, and so on. So there are different types of PostgreSQL. So there are different SQL databases out there. And you can, in this case, we are using SQLite. Okay. And you can, in this case, we are using SQLite. Okay. That's why we are importing SQLite 3, which is the library we are going to use for working with SQLite. All right. Let's see what we are doing here. So first, create a connection object to the SQLite file. So this is your database, this strange name here, followed by SQLite. This is your database name. Okay. So what you want to do is that you want to open a connection with your database. That means this file, the SQLite file. And then, for example, read the data, read the contents of the tables, or maybe write something into the tables, depending on what you want to do. But in our case, we are doing data analysis, so we're not interested in writing something into the database, but just reading the contents. So the next step is to create a cursor object to execute SQL commands on the database. So we need a handle to work with the database. How do we do that? Just call this cursor function here, or the constructor for the cursor. And what this is going to do is that it's going to create a cursor object for you and you can name it whatever you like. In this case, we named it Fires Underline Cursor, C-U-R. So Fires C-U-R.execute is the next line where we execute a query on the database. So we specify the database and then we created a

connection object. Then we use the connection object to create a cursor to run the queries. And now it's time to run queries. So select name. This is your query. Select name from SQLite master where type equals table. SQLite master where type equals table. So what this one does is that it extracts the name from this particular table, SQLite master, which is a special table in SQLite that contains information about the structure of the database. It is created automatically when a new database is created and it contains one rule for each table. So what you're extracting, you're extracting the names of the tables in the database by executing this line. Okay. And then you are fetching all the rows returned by the SQL command and printing them. So fetch all is going to extract all the words. And this is what you get when you run this cell or these lines of code. This is your output on the right-hand side. You get this. Right? That is the list of the tables in the database. OK, now you have the list of the tables, you might want to execute queries on some of the tables. Right. You can do that. So here's an example of getting the table information. You can do that by executing pragma. So that's a keyword. Pragma is a keyword, just the same way. We're using capital letters for the keywords here so you know the difference, but this is not required. If you don't use capital letters, this query still works. But this is a convention that we use capital letters for the keywords in SQL commands to make it more understandable. In this case, select is a keyword, okay? So it must be select, otherwise it will have different meaning. And name is the name of the table or the tables that are going to be retrieved. And from is another keyword. So select from is like the template of the command and then you're filling in the, providing the values to get what you want. Same story here. Pragma is a keyword. So pragma table info fires what this does is that it executes a SQL pragma command to retrieve information about the columns in the fires table of the SQL or SQLite database connected to by the cursor that we created.

01:28:34

We created the cursor here, remember? Fires connection is the connection object that we created. And then we use cursor to construct a cursor. And now we're using the same cursor. So we use the same cursor here to use, to execute this query and fetch the results. We can use the same cursor here to run another query. So execute pragma table info first. And then basically fires is the name of the table. So we're extracting the information about this table fires and pragma of course is the keyword that you're using in this command template. The next step is to fetch the results. So results that equals fires cursor.fetchall. So every time you executed a command, the next step

p is to extract the results. Now that you executed the command, you want to get some data, you want to get some output. So that output, you can retrieve it by calling fetchall. That will fetch all of the data that has been collected as a result of the execution of this query here. And here we are printing out the result set. So as you can see, the result set is, so it will fetch all the rows from the result set as a list of tuples, and each tuple represents one column in the table. So each tuple here, this is each tuple represents one column in the table so each tuple here this is one tuple this represents one column this is another tuple this one represents another column okay it tells you information about the column says for example what is the column index what is the name of the column what is the type of the column, what is the type of the column. In this case, this one, this column is integer, this column is int32, this column is text with the length of 100. So it tells you what is the type of data that's being stored in the column. It tells you whether it's nullable or not. So you can actually see what each parameter means here. The next slide. So CID is the first one, the column ID, the name, then type, then not null means that whether the column is flagged as not null, which means the value should never be null. And this is for, this can be used for checking integrity in your database. Then you have another value here, and this can be used for checking integrity in your database. Then you have another value here, default value, the default value for the column. If there is a default value, and then the primary key says whether that column is used as the primary key or as part of the primary key, because sometimes we use a combination of the columns for the primary key, because sometimes we use a combination of the columns for the primary key, but we cannot find a column that can be used individually as a primary key. Sometimes we use a combination of column. So if the column is used as part of the primary key or is the primary key, this parameter will be set to one. So in this case, the first column, the object ID is either the primary key or part of the primary key. You should look at the rest to see if there's another one that is nominated as primary key or not. Yeah, so this gives you information about, so this way we learn how to extract metadata about a database. So there's a database online. We can extract it, and we can understand what tables are there, what is the structure of the table, what is in the table, what kind of data is there. So we did all that by executing these two commands, these two queries, the select query and then, you know, this pragma table info. And then we can get this information. And from there, we can move on. We might want to, you know, extract part of the data into a data frame and work with it. So this will give us, you know, some good stuff. OK. Import the data from a query into a data frame. Okay. So thi

s is a time that we can import the results of the query, the results that are returned by executing the query into a data frame. For doing that here, we are defining a query, query select state, fire year, blah, blah, date time. So this, what it does is converts the discovery date into a date time format. And so we're casting or converting the current ticket, converting it to a format that we want. We could have done this later as well, but we're doing it when we're executing. And this new column will be called discovery date, which is exactly the same as the original column, which means if this one is going to replace the original column, okay? So if we didn't want to replace the original column, we could have used a different name here for the column that contains the converted discovery date. And then we could have kept both columns in the table. In this case, we are not interested in that. We just replace the discovery date as a datetime object, as a string. We're converting that into a datetime object.

01:34:06

All right. So what this line does, essentially, in summary, it executes a query on the database that selects some fields from database, some columns from database. And from this particular table from the database, so from table fires. OK, so if you look at the structure of table fires, you already know what kind of data is there, you already know there's object ID, there is FOI, FOD ID, there is, there's all these different columns that you can go through. So some of these columns are used here to extract the data from the table, right? So select blah, blah, and we can just provide this query as the input for read SQL query. So this is one input and the second input is Firescon, which is, remember what Firescon was? The object, the database object or database connection object. So this is what we created here. OK, the connection object. So we're providing the connection object and we are providing the query. And what happens inside the read SQL query from Pandas? It pretty much does what we did here. Similar to what we did here, it's going to create a cursor and then execute the command and then return the result. So it's just like, it's only masking it from us and just makes it easier for us to just grab the data from the table. So just specify your query and provide the object, the connection object, you get the data from the table. So just by your query and provide the object, the connection object, you get the data that you need. That's the data frame, work with it, nice and sweet. All right, last part, get and explore metadata, or not exactly last part, almost last part. Get and explore the metadata of a starter file. So this is also straightforward, very similar to what we explored earlier for different files. The approach and method is similar. Just going to quickly go th

rough that, but you can spend some more time going through the examples, execute them and see what happens. So when you execute the script that I have provided in the examples it's going to download all these uh data uh data sets for you or data files for you and then you can that includes this file and then all you need is to just execute the example file or example cells and one by one and see what happens. So here we are providing a file name that is a Stata file. We explained the Stata file is a file that comes from statistical analysis framework tools. statistical analysis framework or tools. And then we can just simply go ahead and read the data using this library, PyReadStat. And what we can do is that we can execute the read data function of PyReadStat and provide the file name. function of pirate stat and provide the file name and specify if it only wants to get the metadata or we want the metadata and the actual data. So if you set it to false, if you set it to true, it's going to only extract the metadata, which is the case in this exam. Then we're printing the type of the gas meta. Oh, by the way, it returns two outputs, okay? So it returns the metadata here, and it also returns the data. In this case, we named this GSS underline empty, which means it's not going to return anything. So we just, this is just a variable, which we're not going to return anything. So we just, this is just a variable which we're not going to use. So we're, but we're using the metadata. We printed the type of the metadata. This is what we get, which is a container, metadata container. And then we can actually extract more information from this container. For example, the number of columns, the number of rows, and column names. So I have commented this out because it's a long list. You can try commenting it and executing it in the cells and see what happens. So that's to get some metadata about your stratifier. So build a data frame from a column description in the metadata. So, yeah, of course, you can extract, you know, different data or metadata from your data file and then create dataframes. So it's just an example of that. Meta calls and meta columns equals pd.dataframe. pd.dataframe obviously creates a dataframe for you. It's calling the dataframe constructor, data equals gssmeta.columnlabels, it's going to extract the column labels and column names, and then you're setting the columns value to be description, so that is to set the column name as description, and then the result of course would be this data frame. So we are creating a column name description that contains the data that we extracted. So we have the labels and we have the names and we are setting the column name to be description. Of course, you know what the head function does.

Import the columns of the data into a data frame. An example, okay, so this is to build the data frame and import the columns of the data into the data frame. So we can actually specify which columns we want to import in this case, year, ID, so on and so forth. So these are the column names that we want to import into the data frame. And then when we read the data, so `pd.readstata`, what it does is that it takes the file path as the input, the columns that need to be read into the data frame. So this is when we don't want to include all the columns. We only are interested in part of the columns, part of the data. We can filter the columns, say we only want these columns and extract those columns. So when we call the `readStata` function, it's going to read the Stata file, but it's not going to include all the data, all the columns, only going to include the columns that we have provided. Okay, so that was another example. And finally, working with JSON file, which will wrap up our lecture as well. I'll show you more examples of this in the future, but quickly, this is the example that I showed you earlier from YouTube data. So if a JSON file, as I said, has multiple levels, you need to read the data first, for example, into a dictionary. And then from dictionary, you can extract what you need and convert it to data frame. So when you have multi-level JSON file, it would not be as straightforward as just, you know, executing a read function and loading it into a data frame. You need to do more of that. You need to do more of that. You need to do more sophisticated processing. As I said, we'll give you more examples, but here's an example of downloading a JSON file from an online source. So this part is easy. I'm just going to quickly skip it because we have already seen examples of how we can read any file from some online resource. So that just read the JSON file for us and stored it somewhere on this. And here we are building a data frame from JSON file. So what we're doing is that we are opening the connection to the JSON file. So this is the name of the JSON file, `shots.json`. We are opening it as JSON data. So we refer to the file as JSON data from this one. So we'd open `shots.json` as `blah, blah`. What it does, it opens the connection to `shots.json` and then gives us a handle to the file that is named `JSON.underlinedata`. So `JSON.load` therefore reads the contents of the JSON file into `shots`, which loads the contents of the JSON file into a dictionary called `shots`. So the output of `json.load` will be a dictionary. How do we know that's a dictionary? We guess no. We can look at the documentation of the function to see what exactly it returns uh if you are not sure then we can just use a variable and then check what type it has returned the first time and so if you check the type of `shots` you will see that's a dictionary and then you can you know how to work with it in the rest of the code. Okay. So, good to cut it short, this will be finished time, but

I'm going to come back to more examples with this in the future lectures. But what you're doing the next steps is to extract the column headers from the headers key in the result set key, so it's multi-level, in the shots dictionary, and to convert the column headers to lowercase using a list comprehension. You have seen these comprehensions before. This is for list comprehension. So you are converting the column headers to lowercase using the list comprehension and assigning them to and then assigning them to column headers. Then you are creating the data frame by the data frame is called shots. So name of your data frame. So create a Pandas data frame called shots with all the rules as the data and column headers as the column names. So based on the information we extracted from the dictionary, now we're creating a data frame. And this on the information we extracted from the dictionary, now we're creating a data frame. And this is the data frame we want. We want a data frame that has all rows as the data and the column headers are the columns of our data frame. That means grid type, game ID, game event ID, all these are the column headers. You can actually go and open the JSON file that will be downloaded into the folder once you execute this piece of code. That means once you execute this part, it will be stored in your local drive. And then you can open this chart.json and look at this structure and see how it looks. You'll see something similar to what we had for the YouTube data. You'll see multiple levels of, multiple levels in your JSON file. And then you can map it into what you can see here. All right, so at this stage, I'm going to stop the recording and I'll be here for further Q&A. If you have any questions, feel free to ask.